

MESSAGE.JAVA

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */

package com.mycompany.message;

import java.util.Scanner;
import java.util.Random;
import java.util.ArrayList;
import java.io.FileWriter;
import java.io.IOException;

public class Message {

    //LOGIN CLASS//

    static class Login {

        private String firstName;

        private String lastName;

        private String username;

        private String password;

        private String cellPhoneNumber;

        public Login(String firstName, String lastName, String username, String password, String
cellPhoneNumber) {

            this.firstName = firstName;

            this.lastName = lastName;

            this.username = username;

        }

    }

}
```

```
    this.password = password;

    this.cellPhoneNumber = cellPhoneNumber;
}
```

```
public boolean checkUserName() {
    return username.contains("_") && username.length() <= 5;
}
```

```
public boolean checkPasswordComplexity() {
    if (password.length() < 8) return false;

    boolean hasCapital = false, hasNumber = false, hasSpecial = false;

    for (char c : password.toCharArray()) {
        if (Character.isUpperCase(c)) hasCapital = true;
        else if (Character.isDigit(c)) hasNumber = true;
        else if (!Character.isLetterOrDigit(c)) hasSpecial = true;
    }

    return hasCapital && hasNumber && hasSpecial;
}
```

```
public boolean checkCellPhoneNumber() {
    //REGEX SOURCED AND ADAPTED FROM://
    //https://www.baeldung.com/java-regex-validate-phone-numbers//
    return cellPhoneNumber.matches("^\\+27\\d{9}$");
}
```

```
public boolean loginUser(String enteredUsername, String enteredPassword) {
    return this.username.equals(enteredUsername) && this.password.equals(enteredPassword);
}
```

```

public String returnLoginStatus(String enteredUsername, String enteredPassword) {
    if (loginUser(enteredUsername, enteredPassword)) {
        return "Welcome " + firstName + " " + lastName + ", it is great to see you again!";
    } else {
        return "Username or password incorrect, please try again.";
    }
}

```

```

public String getFirstName() { return firstName; }
public String getLastName() { return lastName; }
}

```

//MESSAGE CLASS//

```

static class MessageData {
    private String messageID;
    private int messageNumber;
    private String recipient;
    private String messageText;
    private String messageHash;
    private String flag;

    static int totalSent = 0;
    static ArrayList<MessageData> sentMessages = new ArrayList<>();
    static ArrayList<MessageData> storedMessages = new ArrayList<>();
    static ArrayList<MessageData> disregardMessages = new ArrayList<>();
    static ArrayList<String> messageHashes = new ArrayList<>();
    static ArrayList<String> messageIDs = new ArrayList<>();

    public MessageData(int messageNumber, String recipient, String messageText) {

```

```
    this.messageNumber = messageNumber;

    this.recipient = recipient;

    this.messageText = messageText;

    this.messageID = generateMessageID();

    this.messageHash = createMessageHash();
}
```

```
private String generateMessageID() {
    Random rand = new Random();

    long id = (long)(rand.nextDouble() * 9_000_000_000L) + 1_000_000_000L;

    return String.valueOf(id);
}
```

```
public boolean checkMessageID() {
    return messageID.length() <= 10;
}
```

```
public String checkRecipientCell() {
    if (recipient.startsWith("+") && recipient.length() <= 13) {
        return "Cell phone number successfully captured.";
    } else {
        return "Cell phone number is incorrectly formatted or does not contain an international code. Please correct the number and try again.";
    }
}
```

```
public String createMessageHash() {
    String[] words = messageText.trim().split("\\s+");

    String firstWord = words[0];
}
```

```

String lastWord = words[words.length - 1];

String idPrefix = messageID.substring(0, Math.min(2, messageID.length()));

return (idPrefix + ":" + messageNumber + ":" + firstWord + lastWord).toUpperCase();
}

```

```

public String checkMessageLength() {
    if (messageText.length() <= 250) {
        return "Message ready to send.";
    } else {
        int over = messageText.length() - 250;
        return "Message exceeds 250 characters by " + over + "; please reduce the size.";
    }
}

```

```

public String sendMessage(int choice) {
    switch (choice) {
        case 1:
            flag = "Sent";
            totalSent++;
            sentMessages.add(this);
            messageHashes.add(messageHash);
            messageIDs.add(messageID);
            return "Message successfully sent.";
        case 2:
            flag = "Disregarded";
            disregardMessages.add(this);
            return "Press 0 to delete the message.";
        case 3:
            flag = "Stored";

```

```

        storedMessages.add(this);

        messageHashes.add(messageHash);

        messageIDs.add(messageID);

        return "Message successfully stored.";

    default:

        return "Invalid option.";

    }

}

```

```

public static String printMessages() {

    if (sentMessages.isEmpty()) return "No messages sent.";

    StringBuilder sb = new StringBuilder();

    for (MessageData m : sentMessages) {

        sb.append("Message ID: ").append(m.messageID)

            .append(" | Hash: ").append(m.messageHash)

            .append(" | Recipient: ").append(m.recipient)

            .append(" | Message: ").append(m.messageText)

            .append("\n");

    }

    return sb.toString();

}

```

```

public static int returnTotalMessages() {

    return totalSent;

}

```

//STORES STORED MESSAGES TO A JSON FILE//

//FILE WRITING APPROACH ADAPTED FROM://

//<https://www.baeldung.com/java-write-to-file>//

```

public static void storeMessage(String filename) {

    StringBuilder json = new StringBuilder();

    json.append("[\n");

    for (int i = 0; i < storedMessages.size(); i++) {

        MessageData m = storedMessages.get(i);

        json.append(" {\n");

        json.append("  \"messageID\": \"\").append(m.messageID).append("\",\n");

        json.append("  \"messageHash\": \"\").append(m.messageHash).append("\",\n");

        json.append("  \"recipient\": \"\").append(m.recipient).append("\",\n");

        json.append("  \"message\": \"\").append(m.messageText).append("\",\n");

        json.append("  \"flag\": \"\").append(m.flag).append("\",\n");

        json.append(" }");

        if (i < storedMessages.size() - 1) json.append(",");

        json.append("\n");

    }

    json.append("]");

    try (FileWriter fw = new FileWriter(filename)) {

        fw.write(json.toString());

        System.out.println("Messages successfully stored to " + filename);

    } catch (IOException e) {

        System.out.println("Error writing JSON file: " + e.getMessage());

    }

}

```

```

public String getMessageID() { return messageID; }

public String getRecipient() { return recipient; }

public String getMessageText() { return messageText; }

public String getMessageHash() { return messageHash; }

```

```
    public String getFlag()    { return flag; }  
}
```

```
//MAIN//
```

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);
```

```
    Login user = null;
```

```
//WELCOME SCREEN TO REGISTER OR LOGIN//
```

```
boolean welcomed = true;
```

```
while (welcomed) {
```

```
    System.out.println("\n WELCOME TO QUICKCHAT ");
```

```
    System.out.println("1) Register");
```

```
    System.out.println("2) Login");
```

```
    System.out.println("3) Exit");
```

```
    System.out.print("Choose option: ");
```

```
    int welcome;
```

```
    try {
```

```
        welcome = Integer.parseInt(scanner.nextLine().trim());
```

```
    } catch (NumberFormatException e) {
```

```
        System.out.println("Invalid option.");
```

```
        continue;
```

```
    }
```

```
    switch (welcome) {
```

```
        case 1:
```

```
            System.out.println("\nREGISTRATION ");
```

```
System.out.print("First Name: ");  
  
String regFirst = scanner.nextLine();
```

```
System.out.print("Last Name: ");  
  
String regLast = scanner.nextLine();
```

```
String regUsername;  
  
while (true) {  
  
    System.out.print("Username: ");  
  
    regUsername = scanner.nextLine();  
  
    Login tempU = new Login(regFirst, regLast, regUsername, "", "+270000000000");  
  
    if (tempU.checkUserName()) {  
  
        System.out.println("Username successfully captured.");  
  
        break;  
  
    } else {  
  
        System.out.println("Username is not correctly formatted; please ensure that your username  
contains an underscore and is no more than five characters in length.");  
  
    }  
  
}
```

```
String regPassword;  
  
while (true) {  
  
    System.out.print("Password: ");  
  
    regPassword = scanner.nextLine();  
  
    Login tempP = new Login(regFirst, regLast, regUsername, regPassword, "+270000000000");  
  
    if (tempP.checkPasswordComplexity()) {  
  
        System.out.println("Password successfully captured.");  
  
        break;  
  
    }  
  
}
```

```
    } else {  
        System.out.println("Password is not correctly formatted; please ensure that the password  
contains at least 8 characters, a capital letter, a number and a special character.");  
    }  
}
```

```
String regCell;  
while (true) {  
    System.out.print("Cell number (+27): ");  
    regCell = scanner.nextLine();  
    Login tempC = new Login(regFirst, regLast, regUsername, regPassword, regCell);  
    if (tempC.checkCellPhoneNumber()) {  
        System.out.println("Cellphone number successfully captured.");  
        break;  
    } else {  
        System.out.println("Cell phone number incorrectly formatted or does not contain international  
code ( +27 ).");  
    }  
}
```

```
user = new Login(regFirst, regLast, regUsername, regPassword, regCell);  
System.out.println("Registration successful! You can now log in.");  
break;
```

case 2:

```
if (user == null) {  
    System.out.println("No user registered. Please register first.");  
    break;  
}
```

```

        System.out.println("\nLOGIN ");

        System.out.print("Username: ");

        String enteredUsername = scanner.nextLine();

        System.out.print("Password: ");

        String enteredPassword = scanner.nextLine();


        System.out.println(user.returnLoginStatus(enteredUsername, enteredPassword));


        if (user.loginUser(enteredUsername, enteredPassword)) {

            welcomed = false;

        }

        break;


        case 3:

            System.out.println("Goodbye!");

            scanner.close();

            return;


        default:

            System.out.println("Invalid option.");

        }

    }

}


//ASK HOW MANY MESSAGES TO SEND//

System.out.print("\nHow many messages would you like to send? ");

int numMessages = 0;

try {

    numMessages = Integer.parseInt(scanner.nextLine().trim());

} catch (NumberFormatException e) {

```

```

        System.out.println("Invalid number. Exiting.");

        scanner.close();

        return;
    }

    //MAIN MENU LOOP//

    boolean running = true;

    while (running) {

        System.out.println("\n WELCOME TO QUICKCHAT");

        System.out.println("1) Send Messages");

        System.out.println("2) Show Recently Sent Messages");

        System.out.println("3) Quit");

        System.out.print("Choose option: ");

        int choice;

        try {

            choice = Integer.parseInt(scanner.nextLine().trim());

        } catch (NumberFormatException e) {

            System.out.println("Invalid option.");

            continue;

        }

        switch (choice) {

            case 1:

                for (int i = 1; i <= numMessages; i++) {

                    System.out.println("\n--- Message " + i + " of " + numMessages + " ---");

                    System.out.print("Recipient cell (+27...): ");

                    String recipient = scanner.nextLine().trim();

```

```

String messageText;

while (true) {

    System.out.print("Message: ");

    messageText = scanner.nextLine();

    if (messageText.length() <= 250) {

        System.out.println("Message ready to send.");

        break;

    } else {

        int over = messageText.length() - 250;

        System.out.println("Message exceeds 250 characters by " + over + "; please reduce the size.");

    }

}

```

```

MessageData msg = new MessageData(i, recipient, messageText);

```

```

System.out.println(msg.checkRecipientCell());

System.out.println("Message ID generated: " + msg.getMessageID());

System.out.println("Message Hash: " + msg.getMessageHash());

```

```

System.out.println("\n1) Send Message");

System.out.println("2) Disregard Message");

System.out.println("3) Store Message to send later");

System.out.print("Choose: ");

```

```

int sendChoice;

try {

    sendChoice = Integer.parseInt(scanner.nextLine().trim());

} catch (NumberFormatException e) {

```

```

        sendChoice = 2;
    }

    System.out.println(msg.sendMessage(sendChoice));

    System.out.println("\n Message Details ");
    System.out.println("Message ID : " + msg.getMessageID());
    System.out.println("Message Hash : " + msg.getMessageHash());
    System.out.println("Recipient : " + msg.getRecipient());
    System.out.println("Message : " + msg.getMessageText());
}

System.out.println("\nTotal messages sent: " + MessageData.returnTotalMessages());

//SAVE STORED MESSAGES TO JSON AFTER EACH SEND SESSION//
if (!MessageData.storedMessages.isEmpty()) {
    MessageData.storeMessage("stored_messages.json");
}
break;

case 2:
    System.out.println("\nComing Soon.");
    break;

case 3:
    System.out.println("Goodbye!");
    running = false;
    break;

```

```
default:
```

```
    System.out.println("Invalid option.");
```

```
}
```

```
}
```

```
scanner.close();
```

```
}
```

```
}
```

MESSAGETEST.JAVA

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/UnitTests/JUnit5TestClass.java to edit this template
 */

package com.mycompany.message;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.BeforeEach;
import static org.junit.jupiter.api.Assertions.*;

public class MessageTest {

    //RESET ALL STATIC STATE BEFORE EVERY TEST//

    @BeforeEach
    public void resetStatics() {
        Message.MessageData.totalSent = 0;
        Message.MessageData.sentMessages.clear();
        Message.MessageData.storedMessages.clear();
        Message.MessageData.disregardMessages.clear();
        Message.MessageData.messageHashes.clear();
        Message.MessageData.messageIDs.clear();
    }
}
```

```
//LOGIN TESTS//
```

```
@Test
```

```
public void testUsernameValid() {
```

```
    //kyl_1 CONTAINS UNDERSCORE AND IS 5 CHARS - SHOULD PASS//
```

```
    Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",  
"+27838968976");
```

```
    assertTrue(user.checkUserName());
```

```
}
```

```
@Test
```

```
public void testUsernameInvalid() {
```

```
    //NO UNDERSCORE AND MORE THAN 5 CHARS - SHOULD FAIL//
```

```
    Message.Login user = new Message.Login("Kyle", "Jackson", "kyle!!!!", "Ch&&sec@ke99!",  
"+27838968976");
```

```
    assertFalse(user.checkUserName());
```

```
}
```

```
@Test
```

```
public void testPasswordValid() {
```

```
    //Ch&&sec@ke99! HAS CAPITAL, NUMBER, SPECIAL - SHOULD PASS//
```

```
    Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",  
"+27838968976");
```

```
    assertTrue(user.checkPasswordComplexity());
```

```
}
```

```
@Test
```

```
public void testPasswordInvalid() {
```

```
    //password IS ALL LOWERCASE - SHOULD FAIL//
```

```
    Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "password", "+27838968976");
```

```
        assertFalse(user.checkPasswordComplexity());  
    }  
}
```

@Test

```
public void testCellValid() {  
    //+27838968976 MATCHES +27 FORMAT - SHOULD PASS//  
    Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",  
"+27838968976");  
    assertTrue(user.checkCellPhoneNumber());  
}
```

@Test

```
public void testCellInvalid() {  
    //08966553 HAS NO INTERNATIONAL CODE - SHOULD FAIL//  
    Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!", "08966553");  
    assertFalse(user.checkCellPhoneNumber());  
}
```

@Test

```
public void testLoginSuccess() {  
    //CORRECT CREDENTIALS - SHOULD RETURN TRUE//  
    Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",  
"+27838968976");  
    assertTrue(user.loginUser("kyl_1", "Ch&&sec@ke99!"));  
}
```

@Test

```
public void testLoginFail() {  
    //WRONG PASSWORD - SHOULD RETURN FALSE//
```

```
Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",  
"+27838968976");
```

```
    assertFalse(user.loginUser("kyl_1", "wrongpassword"));  
}
```

@Test

```
public void testLoginStatusSuccess() {  
    //CORRECT CREDENTIALS - SHOULD RETURN WELCOME MESSAGE//  
    Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",  
"+27838968976");  
    assertEquals(  
        "Welcome Kyle Jackson, it is great to see you again!",  
        user.returnLoginStatus("kyl_1", "Ch&&sec@ke99!")  
    );  
}
```

@Test

```
public void testLoginStatusFail() {  
    //WRONG CREDENTIALS - SHOULD RETURN FAILURE MESSAGE//  
    Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",  
"+27838968976");  
    assertEquals(  
        "Username or password incorrect, please try again.",  
        user.returnLoginStatus("kyl_1", "wrongpassword")  
    );  
}
```

//MESSAGE LENGTH TESTS//

@Test

```

public void testMessageLengthValid() {

    //SHORT MESSAGE - SHOULD RETURN READY TO SEND//

    Message.MessageData msg = new Message.MessageData(1, "+27718693002", "Hi Mike, can you join us
for dinner tonight?");

    assertEquals("Message ready to send.", msg.checkMessageLength());

}

```

@Test

```

public void testMessageLengthTooLong() {

    //260 CHARS IS 10 OVER LIMIT - SHOULD RETURN ERROR WITH OVERCOUNT//

    String longMsg = "A".repeat(260);

    Message.MessageData msg = new Message.MessageData(1, "+27718693002", longMsg);

    assertTrue(msg.checkMessageLength().contains("exceeds 250 characters by 10"));

}

```

//RECIPIENT TESTS//

@Test

```

public void testRecipientValid() {

    //VALID +27 NUMBER - SHOULD RETURN SUCCESS//

    Message.MessageData msg = new Message.MessageData(1, "+27718693002", "Hi Mike, can you join us
for dinner tonight?");

    assertEquals("Cell phone number successfully captured.", msg.checkRecipientCell());

}

```

@Test

```

public void testRecipientInvalid() {

    //NO INTERNATIONAL CODE - SHOULD RETURN FAILURE//

    Message.MessageData msg = new Message.MessageData(2, "08575975889", "Hi Keegan, did you receive
the payment?");

```

```
        assertTrue(msg.checkRecipientCell().contains("incorrectly formatted or does not contain an  
international code"));  
    }
```

```
//MESSAGE HASH TESTS//
```

```
@Test
```

```
public void testMessageHashUppercase() {  
    //HASH MUST BE ALL UPPERCASE//  
    Message.MessageData msg = new Message.MessageData(1, "+27718693002", "Hi Mike, can you join us  
for dinner tonight?");  
    String hash = msg.getMessageHash();  
    assertEquals(hash, hash.toUpperCase());  
}
```

```
@Test
```

```
public void testMessageHashFormat() {  
    //HASH MUST CONTAIN COLONS AND CORRECT WORD COMBINATION//  
    Message.MessageData msg = new Message.MessageData(1, "+27718693002", "Hi Mike, can you join us  
for dinner tonight?");  
    assertTrue(msg.getMessageHash().contains(":1:HITONIGHT?"));  
}
```

```
//SENTMESSAGE OPTION TESTS//
```

```
@Test
```

```
public void testSendMessageSend() {  
    //CHOICE 1 - SHOULD RETURN SENT MESSAGE//  
    Message.MessageData msg = new Message.MessageData(1, "+27718693002", "Hi Mike, can you join us  
for dinner tonight?");
```

```
    assertEquals("Message successfully sent.", msg.sendMessage(1));  
}
```

@Test

```
public void testSendMessageDisregard() {  
    //CHOICE 2 - SHOULD RETURN DISREGARD MESSAGE//  
    Message.MessageData msg = new Message.MessageData(2, "08575975889", "Hi Keegan, did you receive  
the payment?");  
    assertEquals("Press 0 to delete the message.", msg.sendMessage(2));  
}
```

@Test

```
public void testSendMessageStore() {  
    //CHOICE 3 - SHOULD RETURN STORED MESSAGE//  
    Message.MessageData msg = new Message.MessageData(1, "+27718693002", "Hi Mike, can you join us  
for dinner tonight?");  
    assertEquals("Message successfully stored.", msg.sendMessage(3));  
}
```

//TOTAL MESSAGES TEST//

@Test

```
public void testReturnTotalMessages() {  
    //SEND TWO MESSAGES - TOTAL SHOULD EQUAL 2//  
    Message.MessageData m1 = new Message.MessageData(1, "+27718693002", "Hi Mike, can you join us  
for dinner tonight?");  
    m1.sendMessage(1);  
    Message.MessageData m2 = new Message.MessageData(2, "+27718693002", "Did you get the cake?");  
    m2.sendMessage(1);  
    assertEquals(2, Message.MessageData.returnTotalMessages());  
}
```

```
}  
  
}
```

OUTPUTS

```
WELCOME TO QUICKCHAT  
1) Register  
2) Login  
3) Exit  
Choose option: 2  
No user registered. Please register first.
```

```
WELCOME TO QUICKCHAT  
1) Register  
2) Login  
3) Exit  
Choose option: 1
```

```
REGISTRATION  
First Name: king  
Last Name: cifer  
Username: c_f3r  
Username successfully captured.  
Password: C1fer.004  
Password successfully captured.  
Cell number (+27): +27123456789  
Cellphone number successfully captured.  
Registration successful! You can now log in.
```

```
WELCOME TO QUICKCHAT  
1) Register  
2) Login  
3) Exit  
Choose option: 2
```

```
LOGIN  
Username: c_f3r  
Password: C1fer.004  
Welcome king cifer, it is great to see you again!
```

1) Send Message
2) Disregard Message
3) Store Message to send later
Choose: 3
Message successfully stored.

Message Details
Message ID : 8931406999
Message Hash : 89:2:ROSEBANKROSEBANK
Recipient : +27987654321
Message : rosebank

--- Message 3 of 88 ---
Recipient cell (+27...): +2711122233334
Message: i am RI
Message ready to send.
Cell phone number is incorrectly formatted or does not contain an international code. Please correct the number and try again.
Message ID generated: 5766456249
Message Hash: 57:3:IRI

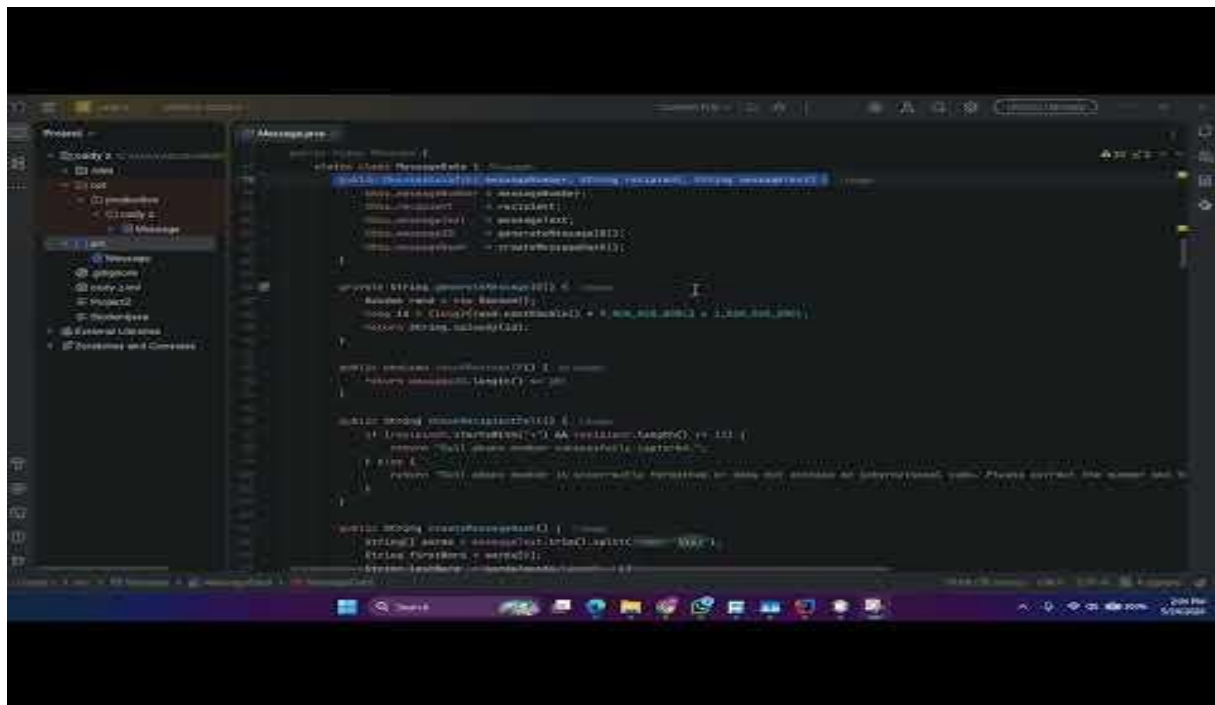
1) Send Message
2) Disregard Message
3) Store Message to send later
Choose: 2
Press 0 to delete the message.

Message Details
Message ID : 5766456249
Message Hash : 57:3:IRI
Recipient : +2711122233334
Message : i am RI

--- Message 4 of 88 ---
Recipient cell (+27...):

GitHub: https://github.com/ST10518408/POE_PART-2

YouTube: <https://youtu.be/zoNyDeT-KFg>



REFERENCES

<https://www.baeldung.com/java-write-to-file>

<https://www.baeldung.com/java-regex-validate-phone-numbers>

<https://docs.oracle.com/javase/tutorial/>

<https://www.jetbrains.com/help/idea/>

<https://www.w3schools.com/java/>